```
form.fields.user.id
# 'u2344999'
```

It's not working! Why not? Because *freezing an object is shallow*—it only freezes the properties and doesn't work recursively to freeze properties of objects that are properties. You must either be careful to freeze the object containing the actual properties you want to be frozen (as you'll see in listing 13.4) or recursively freeze everything in the object to achieve a deep freeze. A typical implementation of a deep freeze follows:

```
deepFreeze = (o) ->
  Object.freeze o
  for own propKey of o
    prop = o[propKey]
    if typeof prop != 'object' || Object.isFrozen prop
      continue
    deepFreeze prop
```

Freezing doesn't follow the prototype chain

Although a frozen object is itself immutable, other objects on its prototype chain may not be. If you're creating mixins or constant objects and you really want them to be isolated, use `null` as the prototype first and then freeze the object.

That's enough time in the deep freeze; what about the other methods, `Object.seal` and `Object.preventExtensions`? They're similar to `Object.freeze`, but they're less restrictive about what can be changed in the object afterward. In table 13.3 you can see the different levels of restriction that these methods place on the objects they're invoked with.

Table 13.3. How `seal`, `freeze`, and `preventExtensions` affect an object